ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

старший преподаватель		Д.О. Шевяков
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

ОРГАНИЗАЦИЯ ПОЛУЧЕНИЯ И ВЫВОДА ДАННЫХ С УДАЛЕННОГО ОБОРУДОВАНИЯ

по курсу: Интернет вещей

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №	4321		И.А. Лаврешин
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Приобретение навыков организации получения данных с удаленного оборудования.

2 Задание и исходные данные

Реализовать эмуляторы вещей для всех классов, созданных в предыдущей лабораторной. Разработать один или несколько интерфейсов, в которые будут передаваться данные от эмуляторов. В рамках данной лабораторной работы необходимо передать в интерфейс только данные для мониторинга. Передача команд, которые осуществляют управление вещью, будет рассмотрена в следующей лабораторной. Для каждой вещи необходимо реализовать собственный ajax-запрос и собственную функцию подключения.

3 Ход выполнения задания

В модуле things.py для классов ScoreSensor, ZoneSensor, EnvSensor, Scoreboard добавлены методы emulate() (случайное обновление полей) и connect(), возвращающие словарь для JSON. Объекты вещей объявлены глобально в app.py. Маршруты GET /monitor/env, /monitor/score, /monitor/zone, /monitor/board вызывают connect() соответствующей вещи. В index.js настроены setInterval и отдельные ajax GET-запросы (jQuery) на каждый маршрут с интервалом 1 с. Шаблон sensor_emulator.html отображает поля только для чтения. Результаты работы представлены на рисунке 1 – 2.

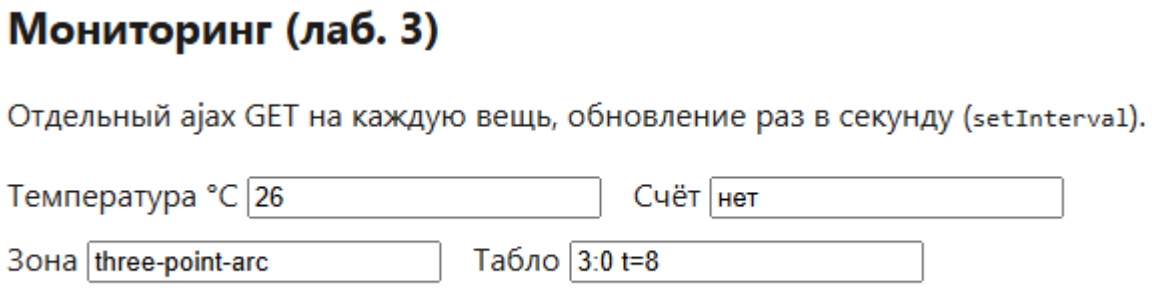

Рисунок 1 – Результаты работы программы

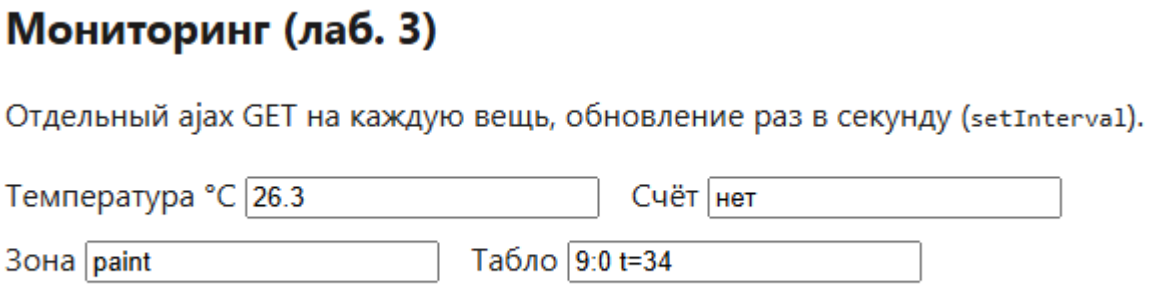

Рисунок 2 – Результаты работы программы

4 Вывод

В ходе выполнения лабораторной работы были успешно реализованы эмуляторы для всех классов вещей, созданных в предыдущей лабораторной работе. Для каждого класса были добавлены методы emulate(), обеспечивающий генерацию случайных данных мониторинга, и connect(), формирующий словарь для последующей передачи данных в формате JSON.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

```
// ============================
// File: sensor_emulator.html
// ============================
<!DOCTYPE html>
<html lang="ru">
  <head>
    <meta charset="utf-8" />
    <title>Эмуляторы — лаб. 3</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/main.css') }}" />
    <script src="https://code.jquery.com/jquery-3.3.1.min.js"></script>
    <script src="{{ url_for('static', filename='js/index.js') }}"></script>
  </head>
  <body>
    <p><a href="{{ url_for('index') }}">← На главную</a></p>
    <h1>Мониторинг (лаб. 3)</h1>
      <p>Отдельный ajax GET на каждую вещь, обновление раз в секунду (<code>setInterval</code>).</p>
    <div class="row">
      <label>Температура °C <input type="text" id="mon_env" readonly /></label>
      <label>Счёт <input type="text" id="mon_score" readonly /></label>
    </div>
    <div class="row">
      <label>Зона <input type="text" id="mon_zone" readonly /></label>
      <label>Табло <input type="text" id="mon_board" readonly /></label>
    </div>
  </body>
</html>
```

```
// ============================
// File: app.py
// ============================
from __future__ import annotations

from flask import Flask, jsonify, render_template

import things

app = Flask(__name__)

env_sensor = things.EnvSensor("es1", "Микроклимат зала", 22.0, 400.0)
score_sensor = things.ScoreSensor("ss1", "Кольцо команды A", is_goal=False)
zone_sensor = things.ZoneSensor("zs1", "Разметка площадки", "three-point-arc")
scoreboard = things.Scoreboard("sb1", "Табло арены", 0, 0, 0)

store = things.CourtDataStore()
mcu = things.MainControlUnit(
    [score_sensor, zone_sensor, env_sensor, scoreboard],
    store,
    scoreboard,
)


def demo_iot_cycle() -> None:
    print("=== collectData ===")
    mcu.collectData()
    print("=== calculateScore ===")
    mcu.calculateScore()
    print("=== updateScoreboard ===")
```

```
    mcu.updateScoreboard()
    print("=== getRecords ===", len(store.getRecords()))


@app.route("/monitor/env")
def monitor_env():
    return jsonify(env_sensor.connect())


@app.route("/monitor/score")
def monitor_score():
    return jsonify(score_sensor.connect())


@app.route("/monitor/zone")
def monitor_zone():
    return jsonify(zone_sensor.connect())


@app.route("/monitor/board")
def monitor_board():
    return jsonify(scoreboard.connect())


@app.route("/lab2")
def lab2_page():
    demo_iot_cycle()
    return render_template("lab2.html")
```

```
@app.route("/")
def index():
    return render_template("index.html")


@app.route("/emulator")
def emulator_page():
    return render_template("sensor_emulator.html")


if __name__ == "__main__":
    app.run(debug=True)

// ============================
// File: things.py
// ============================
from __future__ import annotations

import abc
import random
from typing import Any, Dict, List, Optional

class Thing(abc.ABC):

    def __init__(self, thing_id: str, name: str) -> None:
        self.id = thing_id
        self.name = name

    @abc.abstractmethod
    def getStatus(self) -> str:
```

```
        pass

    def emulate(self) -> None:
        pass

    def connect(self, request: Any = None) -> Dict[str, Any]:
        raise NotImplementedError


class ScoreSensor(Thing):
    def __init__(self, thing_id: str, name: str, is_goal: bool = False) -> None:
        super().__init__(thing_id, name)
        self.is_goal = is_goal

    def getStatus(self) -> str:
        return "ScoreSensor[" + self.id + "]: is_goal=" + str(self.is_goal)

    def emulate(self) -> None:
        self.is_goal = random.choice([True, False])

    def connect(self, request: Any = None) -> Dict[str, Any]:
        self.emulate()
        return {"id": self.id, "value": 1 if self.is_goal else 0, "is_goal": self.is_goal}


class ZoneSensor(Thing):
    ZONES = ("three-point-arc", "paint", "midcourt", "baseline")

    def __init__(self, thing_id: str, name: str, zone: str) -> None:
        super().__init__(thing_id, name)
```

```
        self.zone = zone

    def getStatus(self) -> str:
        return "ZoneSensor[" + self.id + "]: zone=" + self.zone

    def emulate(self) -> None:
        self.zone = random.choice(self.ZONES)

    def connect(self, request: Any = None) -> Dict[str, Any]:
        self.emulate()
        return {"id": self.id, "value": self.zone, "zone": self.zone}


class EnvSensor(Thing):
    def __init__(self, thing_id: str, name: str, temperature: float, light_lux: float) ->
None:
        super().__init__(thing_id, name)
        self.temperature = float(temperature)
        self.lightLux = float(light_lux)

    def getStatus(self) -> str:
        return (
            "EnvSensor["
            + self.id
            + "]: t="
            + str(round(self.temperature, 1))
            + ", lux="
            + str(int(self.lightLux))
        )
```

```
    def emulate(self) -> None:
        self.temperature = round(random.uniform(18.0, 28.0), 1)
        self.lightLux = float(random.randint(300, 600))

    def connect(self, request: Any = None) -> Dict[str, Any]:
        self.emulate()
        return {
            "id": self.id,
            "value": self.temperature,
            "temperature": self.temperature,
            "lightLux": int(self.lightLux),
        }


class Scoreboard(Thing):
    def __init__(self, thing_id: str, name: str, score_a: int = 0, score_b: int = 0,
timer_sec: int = 0) -> None:
        super().__init__(thing_id, name)
        self.scoreA = score_a
        self.scoreB = score_b
        self.timerSec = timer_sec

    def getStatus(self) -> str:
        return (
            "Scoreboard["
            + self.id
            + "]: "
            + str(self.scoreA)
            + ":"
            + str(self.scoreB)
```

```
            + ", t="
            + str(self.timerSec)
        )

    def emulate(self) -> None:
        self.timerSec += random.randint(0, 2)
        if random.random() < 0.3:
            self.scoreA += 1

    def connect(self, request: Any = None) -> Dict[str, Any]:
        self.emulate()
        return {
            "id": self.id,
            "value": self.scoreA,
            "scoreA": self.scoreA,
            "scoreB": self.scoreB,
            "timer": self.timerSec,
        }


class CourtDataStore:
    def __init__(self) -> None:
        self.events: List[str] = []
        self.telemetry: List[str] = []

    def saveEvent(self, data: Any) -> None:
        self.events.append("event=" + repr(data))

    def saveTelemetry(self, data: Any) -> None:
        self.telemetry.append("telemetry=" + repr(data))
```

```
    def getRecords(self) -> List[str]:
        return self.events + self.telemetry


class MainControlUnit:
    def __init__(self, things: List[Thing], store: CourtDataStore, scoreboard: Scoreboard) -> None:
        self.things = things
        self.store = store
        self.scoreboard = scoreboard

    def collectData(self) -> None:
        for t in self.things:
            self.store.saveTelemetry({"id": t.id, "status": t.getStatus()})

    def calculateScore(self) -> None:
        for t in self.things:
            if isinstance(t, ScoreSensor) and t.is_goal:
                self.scoreboard.scoreA += 1
                self.store.saveEvent({"type": "basket", "points": self.scoreboard.scoreA})
                break

    def updateScoreboard(self) -> None:
        self.scoreboard.timerSec += 1

// ============================
// File: index.js
// ============================
```

```
function poll(url, id, pick) {
  $.ajax({
    type: "GET",
    url: url,
    dataType: "json",
    contentType: "application/json",
    data: {},
    success: function (response) {
      document.getElementById(id).value = pick(response);
    },
  });
}

$(function () {
  setInterval(function () {
    poll("/monitor/env", "mon_env", function (r) {
      return r.temperature != null ? r.temperature : r.value;
    });
  }, 1000);
  setInterval(function () {
    poll("/monitor/score", "mon_score", function (r) {
      return r.is_goal ? "гол" : "нет";
    });
  }, 1000);
  setInterval(function () {
    poll("/monitor/zone", "mon_zone", function (r) {
      return r.zone;
    });
  }, 1000);
  setInterval(function () {
```

```
    poll("/monitor/board", "mon_board", function (r) {
      return r.scoreA + ":" + r.scoreB + " t=" + r.timer;
    });
  }, 1000);
});
```